Tutorial – 1

Code

```
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# ------------------------------------------------------------------------
# 1. Initialization and Setup
# ------------------------------------------------------------------------
print("Downloading required NLTK data...")
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('vader_lexicon', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
print("Downloads complete!\n")

# ------------------------------------------------------------------------
# 2. Sample Data
# ------------------------------------------------------------------------
# 5 diverse sample paragraphs with known sentiments
# 1 = Positive, 0 = Negative
samples = [
    {
        "text": "The movie was absolutely fantastic! The acting was great and the plot kept me engaged until the very end. I highly recommend it 10/10.",
        "label": 1
    },
    {
        "text": "Terrible experience at the restaurant. The food was cold, the service was incredibly slow, and the prices were way too high for such poor quality.",
        "label": 0
    },
    {
        "text": "I really loved the new software update. It fixed all the bugs and the new features are amazingly useful and intuitive.",
        "label": 1
    },
    {
        "text": "What a complete waste of time. The book was boring, the characters were flat, and the ending made absolutely no sense at all.",
        "label": 0
    },
    {
        "text": "The customer support team was very helpful and resolved my issue within minutes. Great service and very friendly staff!",
        "label": 1
    }
]

# ------------------------------------------------------------------------
# Functions for NLP Pipeline
# ------------------------------------------------------------------------

def clean_text(text):
    """Perform text cleaning and normalization."""
    # Convert to lowercase
    text = text.lower()
    # Remove special characters and numbers (keep only alphabets and spaces)
    text = re.sub(r'[^a-z\s]', '', text)
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def tokenize_text(text):
    """Apply word tokenization."""
    return word_tokenize(text)
```

```python
def remove_stopwords(tokens):
    """Remove stop words using NLTK's English stop words."""
    stop_words = set(stopwords.words('english'))
    return [word for word in tokens if word not in stop_words]

def apply_stemming(tokens):
    """Apply stemming using PorterStemmer."""
    stemmer = PorterStemmer()
    return [stemmer.stem(word) for word in tokens]

def apply_lemmatization(tokens):
    """Apply lemmatization using WordNetLemmatizer."""
    lemmatizer = WordNetLemmatizer()
    return [lemmatizer.lemmatize(word) for word in tokens]

# Initialize VADER sentiment analyzer
sia = SentimentIntensityAnalyzer()

def analyze_sentiment(text):
    """Classify the text as Positive (1) or Negative (0) using VADER."""
    # VADER returns a compound score between -1 and 1
    scores = sia.polarity_scores(text)
    # We use a simple threshold: > 0 is positive, otherwise negative
    return 1 if scores['compound'] > 0 else 0

# ------------------------------------------------------------------------------
# 3. Processing and Evaluation
# ------------------------------------------------------------------------------

actual_labels = []
predicted_labels = []

print("="*60)
print(" NLP Text Evaluation System - Processing Pipeline")
print("="*60)

for i, sample in enumerate(samples):
    original_text = sample["text"]
    actual_label = sample["label"]
    actual_labels.append(actual_label)

    print(f"\n--- Document {i+1} ---")
    print(f"Original Text:\n  {original_text}")
    print("-" * 40)

    # Step 1: Cleaning
    cleaned_text = clean_text(original_text)
    print(f"1. Cleaned Text:\n  {cleaned_text}")

    # Step 2: Tokenization
    tokens = tokenize_text(cleaned_text)
    print(f"2. Tokenized:\n  {tokens}")

    # Step 3: Stopword Removal
    filtered_tokens = remove_stopwords(tokens)
    print(f"3. Stopwords Removed:\n  {filtered_tokens}")

    # Step 4: Stemming
    stemmed_tokens = apply_stemming(filtered_tokens)
    print(f"4. Stemmed Tokens:\n  {stemmed_tokens}")

    # Step 5: Lemmatization
    lemmatized_tokens = apply_lemmatization(filtered_tokens)
    print(f"5. Lemmatized Tokens:\n  {lemmatized_tokens}")

    # Note: For sentiment analysis with VADER, it's often better to use
    # the original or lightly cleaned text as it relies on punctuation and case.
    # However, to use the processed output, we'll join the lemmatized tokens back.
    processed_text_for_vader = " ".join(lemmatized_tokens)

    # Step 6: Sentiment Classification
    predicted_label = analyze_sentiment(processed_text_for_vader)
    predicted_labels.append(predicted_label)

    label_map = {1: "Positive", 0: "Negative"}
    print(f"\nClassification Result:")
    print(f"  Actual: {label_map[actual_label]} | Predicted: {label_map[predicted_label]}")
    print("="*60)
```

```python
# ------------------------------------------------------------------------
# 4. Final Evaluation Results
# ------------------------------------------------------------------------

print("\n" + "="*60)
print(" Final Evaluation Metrics")
print("="*60)

# Generate confusion matrix
cm = confusion_matrix(actual_labels, predicted_labels)
print("\nConfusion Matrix:")
print("                 Predicted Negative  Predicted Positive")
print(f"Actual Negative       {cm[0][0]}                  {cm[0][1]}")
print(f"Actual Positive       {cm[1][0]}                  {cm[1][1]}")

# Calculate metrics
# pos_label=1 means we are treating "Positive" as the positive class
precision = precision_score(actual_labels, predicted_labels, pos_label=1)
recall = recall_score(actual_labels, predicted_labels, pos_label=1)
f1 = f1_score(actual_labels, predicted_labels, pos_label=1)

print("\nPerformance Metrics:")
print(f"  Precision: {precision:.4f}")
print(f"  Recall:    {recall:.4f}")
print(f"  F1-Score:  {f1:.4f}")
print("="*60)
```

Output

```
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials> python '.\Tutorial - 1\main.py'
Downloading required NLTK data...
Downloads complete!

============================================================
 NLP Text Evaluation System - Processing Pipeline
============================================================

--- Document 1 ---
Original Text:
  The movie was absolutely fantastic! The acting was great and the plot kept me engaged until the very end. I highly recommend it 10/10.
----------------------------------------
1. Cleaned Text:
  the movie was absolutely fantastic the acting was great and the plot kept me engaged until the very end i highly recommend it
2. Tokenized:
  ['the', 'movie', 'was', 'absolutely', 'fantastic', 'the', 'acting', 'was', 'great', 'and', 'the', 'plot', 'kept', 'me', 'engaged', 'until', 'the', 'very',
 'end', 'i', 'highly', 'recommend', 'it']
3. Stopwords Removed:
  ['movie', 'absolutely', 'fantastic', 'acting', 'great', 'plot', 'kept', 'engaged', 'end', 'highly', 'recommend']
4. Stemmed Tokens:
  ['movi', 'absolut', 'fantast', 'act', 'great', 'plot', 'kept', 'engag', 'end', 'highli', 'recommend']
5. Lemmatized Tokens:
  ['movie', 'absolutely', 'fantastic', 'acting', 'great', 'plot', 'kept', 'engaged', 'end', 'highly', 'recommend']

Classification Result:
  Actual: Positive | Predicted: Positive
============================================================

--- Document 2 ---
Original Text:
  Terrible experience at the restaurant. The food was cold, the service was incredibly slow, and the prices were way too high for such poor quality.
----------------------------------------
1. Cleaned Text:
  terrible experience at the restaurant the food was cold the service was incredibly slow and the prices were way too high for such poor quality
2. Tokenized:
  ['terrible', 'experience', 'at', 'the', 'restaurant', 'the', 'food', 'was', 'cold', 'the', 'service', 'was', 'incredibly', 'slow', 'and', 'the', 'prices',
 'were', 'way', 'too', 'high', 'for', 'such', 'poor', 'quality']
3. Stopwords Removed:
  ['terrible', 'experience', 'restaurant', 'food', 'cold', 'service', 'incredibly', 'slow', 'prices', 'way', 'high', 'poor', 'quality']
4. Stemmed Tokens:
  ['terribl', 'experi', 'restaur', 'food', 'cold', 'servic', 'incred', 'slow', 'price', 'way', 'high', 'poor', 'qualiti']
5. Lemmatized Tokens:
  ['terrible', 'experience', 'restaurant', 'food', 'cold', 'service', 'incredibly', 'slow', 'price', 'way', 'high', 'poor', 'quality']

Classification Result:
  Actual: Negative | Predicted: Negative
============================================================

--- Document 3 ---
Original Text:
  I really loved the new software update. It fixed all the bugs and the new features are amazingly useful and intuitive.
----------------------------------------
1. Cleaned Text:
  i really loved the new software update it fixed all the bugs and the new features are amazingly useful and intuitive
2. Tokenized:
  ['i', 'really', 'loved', 'the', 'new', 'software', 'update', 'it', 'fixed', 'all', 'the', 'bugs', 'and', 'the', 'new', 'features', 'are', 'amazingly', 'us
eful', 'and', 'intuitive']
3. Stopwords Removed:
  ['really', 'loved', 'new', 'software', 'update', 'fixed', 'bugs', 'new', 'features', 'amazingly', 'useful', 'intuitive']
4. Stemmed Tokens:
  ['realli', 'love', 'new', 'softwar', 'updat', 'fix', 'bug', 'new', 'featur', 'amazingli', 'use', 'intuit']
5. Lemmatized Tokens:
  ['really', 'loved', 'new', 'software', 'update', 'fixed', 'bug', 'new', 'feature', 'amazingly', 'useful', 'intuitive']

Classification Result:
  Actual: Positive | Predicted: Positive
============================================================

--- Document 4 ---
Original Text:
  What a complete waste of time. The book was boring, the characters were flat, and the ending made absolutely no sense at all.
----------------------------------------
1. Cleaned Text:
  what a complete waste of time the book was boring the characters were flat and the ending made absolutely no sense at all
2. Tokenized:
  ['what', 'a', 'complete', 'waste', 'of', 'time', 'the', 'book', 'was', 'boring', 'the', 'characters', 'were', 'flat', 'and', 'the', 'ending', 'made', 'abs
olutely', 'no', 'sense', 'at', 'all']
3. Stopwords Removed:
  ['complete', 'waste', 'time', 'book', 'boring', 'characters', 'flat', 'ending', 'made', 'absolutely', 'sense']
4. Stemmed Tokens:
  ['complet', 'wast', 'time', 'book', 'bore', 'charact', 'flat', 'end', 'made', 'absolut', 'sens']
5. Lemmatized Tokens:
  ['complete', 'waste', 'time', 'book', 'boring', 'character', 'flat', 'ending', 'made', 'absolutely', 'sense']

Classification Result:
  Actual: Negative | Predicted: Negative
============================================================

--- Document 5 ---
Original Text:
  The customer support team was very helpful and resolved my issue within minutes. Great service and very friendly staff!
----------------------------------------
1. Cleaned Text:
  the customer support team was very helpful and resolved my issue within minutes great service and very friendly staff
2. Tokenized:
  ['the', 'customer', 'support', 'team', 'was', 'very', 'helpful', 'and', 'resolved', 'my', 'issue', 'within', 'minutes', 'great', 'service', 'and', 'very',
 'friendly', 'staff']
3. Stopwords Removed:
  ['customer', 'support', 'team', 'helpful', 'resolved', 'issue', 'within', 'minutes', 'great', 'service', 'friendly', 'staff']
4. Stemmed Tokens:
  ['custom', 'support', 'team', 'help', 'resolv', 'issu', 'within', 'minut', 'great', 'servic', 'friendli', 'staff']
5. Lemmatized Tokens:
  ['customer', 'support', 'team', 'helpful', 'resolved', 'issue', 'within', 'minute', 'great', 'service', 'friendly', 'staff']

Classification Result:
  Actual: Positive | Predicted: Positive
============================================================

============================================================
 Final Evaluation Metrics
============================================================

Confusion Matrix:
                 Predicted Negative  Predicted Positive
Actual Negative        2                   0
Actual Positive        0                   3

Performance Metrics:
  Precision: 1.0000
  Recall:    1.0000
  F1-Score:  1.0000
============================================================
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials>
```